

Voice Card

Technical, direct, plain, no buzzwords.

Before/After

Before (AI Generic): I am a Backend AI Engineer focused on building robust, scalable AI pipelines that transform complex, unstructured data into actionable intelligence. Whether it's genomic research or financial security, my goal is to bridge the gap between cutting-edge AI models and production-grade reliability.

After (My Voice): I'm a backend AI engineer. I build AI pipelines that turn messy, unstructured data into something reliable enough to use in production—whether that's genomic research or financial fraud detection. My focus is closing the gap between a model that works in a notebook and a system that works in the real world.

Genomic-RAG Pipeline for RARS1 Variant Analysis

The problem

Research on the RARS1 gene is scattered across hundreds of papers and abstracts on PubMed. If you wanted to know what a specific variant actually does, you had to read through them one by one. That's slow, and it's easy to miss something important.

There was no way to search this literature directly and get a straight, evidence-backed answer. I built one: a pipeline that turns the literature into a searchable vector database and answers natural-language questions using only what's actually in the source abstracts.

What I built

Data collection. Searching PubMed for just "RARS1" barely returned anything — 12 articles, not enough to work with. I built four query variations (RARS1, arginyl-tRNA synthetase, ArgRS, RARS, cytoplasmic), pulled 200 articles per query, then ran a custom filter on titles and abstracts to narrow it down to the 50 most relevant.

Chunking. Standard chunking tools split HGVS mutation names right down the middle — c.5A>G would come out as two meaningless fragments. I wrote a custom regex-based chunker that keeps these intact.

Vector database. ChromaDB, with all-MiniLM-L6-v2 for embeddings. It's fast, runs locally, and handles the semantic relationships in biological text well.

LLM. I started with OpenAI, but rate limits, API errors, and inconsistent JSON output made it unreliable for the structured variant/disease data I needed. I switched to Qwen2.5-3B running

locally on Ollama. Zero API cost, and fully reproducible since nothing depends on an external service.

Guardrail against hallucination. If a question falls outside what the retrieved context actually supports, the system says “I do not know” instead of guessing. Every answer has to come with a PMID source.

What didn’t work

- **Narrow search.** My first PubMed search using just “RARS1” returned only 12 articles — nowhere near enough. I had to broaden the query strategy significantly before the system had anything real to work with.
- **OpenAI as the backbone.** Rate limits and inconsistent structured output slowed the whole project down and made it hard to trust. Moving to a local model fixed both the cost and the stability problem.
- **Standard chunking.** LangChain’s default chunking broke mutation names like c.2T>C (p.Met1Thr) in half, which meant the system couldn’t recognize them as genetic data at all. I had to write my own chunking logic to keep that structure intact.

What it actually does

I tested it two ways.

Hallucination tests — asking questions it shouldn’t be able to answer: - “COVID-19 symptoms?” → “I do not know based on the retrieved literature.” Pass. - “Diabetes treatment?” → Same. Pass. - “RARS2 variants?” → It correctly identified RARS2 but refused to connect it to RARS1. Pass.

Real query test — asking what it’s actually for: - Query: “List the genetic variants of RARS1 and their associated clinical phenotypes and diseases with PMIDs.” - Result: for the c.5A>G variant, it correctly linked phenotypes like hypomyelination and nystagmus to Pelizaeus-Merzbacher-like disease, with PMID:33515434 cited for the claim, in structured JSON.

It doesn’t just generate plausible-sounding text. It stays inside the evidence and shows its source every time.

What I’d change next

1. **Hybrid search.** Right now it’s pure vector search. For specific terms like mutation names, keyword search (BM25) would do better. Combining vector and keyword search would improve accuracy on exact matches.
 2. **Reranking.** I currently take the top 50 chunks by vector similarity, but the most useful one for a given question isn’t always the highest-scoring one. Adding a reranking layer (like Cohere Rerank) would improve what actually reaches the model.
 3. **Automated evaluation.** My testing so far has been manual — trick questions and spot checks. I’d bring in a framework like RAGAS to track faithfulness and relevance numerically, so I can measure not just whether it works, but how consistent it is.
-

Real-Time Credit Card Fraud Detection Pipeline

The problem

Fraud cases in credit card transaction data are rare — about 0.1% to 0.2% of all transactions. That sounds small, but it breaks standard models. A model can label every single transaction as “legitimate” and still hit over 99% accuracy, while catching zero actual fraud. That’s a false sense of success.

My goal was to build a system that gets past this trap and actually detects fraud, not just optimizes a misleading number.

What I built

Data & preprocessing. I used the standard credit card fraud dataset. To deal with the class imbalance, I applied SMOTE to generate synthetic samples for the fraud class, so the model would actually learn fraud patterns instead of defaulting to “legit” every time.

Model architecture. I moved past simple classifiers to a Deep ANN. I added Dropout layers to prevent overfitting and Batch Normalization to stabilize training. This handled the complexity of the feature space better than a decision tree could.

Pipeline & deployment. Built end-to-end: - FastAPI for inference requests. - Redis as a caching layer, bringing latency down to ~50ms — important for a real-time financial system. - A Streamlit dashboard for monitoring live traffic and triggering alerts on high-risk transactions.

Evaluation. I dropped accuracy as a metric — it’s misleading here. I optimized for Precision-Recall AUC and Recall instead, since missing actual fraud costs far more than a false positive.

What didn’t work

- **Aggressive undersampling.** My first approach balanced the classes to 50/50 by undersampling. It worked on paper, but the model lost so much information from the legitimate-transaction majority that it started flagging real transactions as fraud constantly. The false positive rate made it unusable.
- **Default threshold (0.5).** When I moved to the Deep ANN, I left the risk threshold at the default 0.5. It caught almost nothing — the model was too conservative. I had to lower the threshold significantly to catch more fraud, accepting a slightly higher false positive rate as the tradeoff.
- **Caching everything in Redis.** I originally planned to cache every single inference result. That was unnecessary overhead and wasted memory for the traffic levels I was simulating. I reworked it to only cache high-frequency features and model metadata, which kept latency low without the waste.

What it actually does

After tuning the threshold, the final numbers:

- **ROC-AUC: 0.958** — strong separation between fraud and legitimate transactions.
- **Threshold set at 0.90:** F1 score of 0.77, Precision 0.738, Recall 0.806. Lower thresholds caught more fraud but spiked false alerts — 0.90 was the most practical operating point.
- **Latency: ~50ms end-to-end**, consistent, thanks to Redis caching and offloading transaction metadata to memory.
- **Built for high-concurrency traffic** — the pipeline stayed performant under simulated peak load.

What I'd change next

1. **Sequence modeling.** Right now the model looks at individual transactions. Adding user behavior data — location changes, spending pattern shifts over the last 24 hours — and moving to an RNN/LSTM or Transformer architecture would let the system evaluate risk across a sequence of transactions, not just one at a time.
2. **Handling concept drift.** Financial behavior changes over time, and model performance degrades as a result. I'd build an incremental learning or automated retraining pipeline so the model keeps updating itself with new data instead of going stale.
3. **Explainability.** Beyond the Streamlit dashboard, I'd add SHAP or LIME so the system can explain *why* it flagged a transaction as fraud, not just that it did. That's the difference between "there's a risk" and something a fraud analyst can actually act on with confidence.

About Me

I'm a backend AI engineer. I build AI pipelines that turn messy, unstructured data into something reliable enough to use in production — whether that's genomic research or financial fraud detection. My focus is closing the gap between a model that works in a notebook and a system that works in the real world.

Contact

- GitHub: github.com/caglaeren
- LinkedIn: linkedin.com/in/cagla-eren